Typescript

Contents

a)Four Examples

15. Object-Oriented Programming

a) Class
b) Object
c) Inheritance
d) Polymorphism
e) Encapsulation
f) Abstraction

16. Class

a) Example of class with variables & functions
b) Example with default constructor
c) Example with parameterized constructor
d) Example 2 with parameterized constructor

17. Inheritance

a) Single level Inheritance
b) Multi Level
c) Multiple
d) Hierarchical
e) Hybrid

18. Interfaces

f) a)Example 1 Define the shape of the object
g) b)Create an object that add to user interface.
h) c)Example 2
i) d)Example 3 Interface with JSON Object
j) e)Example 4 methods with the interface

19. Single level Inheritance in Interface

20. Multi Level Inheritance in Interface

21. Multiple Inheritance in Interface

22. Abstract Classes

23 Access Modifiers

a) Public
b) Protected

c) Private

24 Polymorphism

a) Method Overriding
b) Method Overloading

25 Advanced Keywords

a) Static Keyword
b) Read-only Keyword
c) Super Keyword
d) Difference between **this** keyword & **Super** Keyword

26 Modules

27 Generics

28 JSON

a) JSON with all Examples
b) Namespace

29 Tuples

30 Enums

- TypeScript is the **client-side programming Language**.
- TypeScript was developed and maintained by **Microsoft.**
- Typescript used to develop Angular, React, Vue.js, and Next.js Applications.
- The current Version of the Typescript is **5.9.3.**
- Typescript follows the **OOPs:**

  (Class, object, inheritance, Polymorphism, Encapsulation, Abstraction).
- Typescript is also called a **superset** of JavaScript.
- The Extension of the TypeScript file is **.ts.**
- VS Code is a suitable IDE for developing TypeScript applications. The browser is unable to understand TypeScript.
- So, as a TypeScript developer, we must convert TypeScript to equivalent JavaScript
- The process of converting ts to js is called the **"Transpilation".**
- **"Tsc"** is the tool used to perform Transpilation.
- "Tsc" stands for the **TypeScript compiler**.
- TypeScript installation is **command based installation**.
- Typescript installation depends on **"npm".**
- "npm" stands **for Node Package Manager.**
- "npm" is present in **Node.js.**

Installation of Typescript:

- Download and install Node.js
- Download and install VS Code.
- Install Typescript
- >npm install –location=global typescript
- tsc –version.

Transpilation

1. The process of converting a TS file into JS is called Transpilation.
2. TypeScript has the OOPs Language. But JS doesn't OOPs language. It is an object-based language.
3. After ES6 the **prototype chaining concept** came indirectly, we can achive the OOPs in Js.
4. But a developer cannot write the complex code.
5. So, we can use TypeScript to generate the JS.

Example:

- TypeScript file ===>tsc demo.ts
- JavaScript file ===> demo.js
- To run the JS file ===>node demo.js.

Variables:

- Variables are used to store the data.
- We can declare variables in 3 ways.

1. Var
2. Let
3. Const

- Let and const are introduced in ES6 Version.
- Variable declaration should contain **a-z, A-Z, 0-9, $ and _** ,
- Variable declaration should not start with **digits or numbers**.
- TypeScript supports the following data types:

1. string
2. number
3. Boolean
4. any
5. never
6. array
7. object
8. tuples
9. type
10. union(|)

Primary (Primitive) Data Types in TypeScript:

Data Type	Description	Example
number	Represents numeric values	let age: number = 25;
string	Represents text values	let name: string = "Sai";
boolean	Represents true or false	let isActive: boolean = true;
null	Represents an empty value	let data: null = null;
undefined	Variable declared but not assigned	let value: undefined;
bigint	Represents very large numbers	let big: bigint = 123n;
symbol	Represents a unique identifier	let id: symbol = Symbol("id");

Special Data Types in TypeScript:

Data Type	Description	Example
any	Allows any type (no type checking)	let data: any = 10;

Non-Primary (Non-Primitive) Data Types in TypeScript

Data Type	Description	Example
array	Stores multiple values	let nums: number[] = [1, 2, 3];
object	Stores key-value pairs	let user = { name: "Sai", age: 25 };
tuple	Fixed-length array with fixed types	let emp: [number, string] = [1, "Dev"];
enum	Collection of named constants	enum Status { Active, Inactive }
function	Block of reusable code	function add(a: number, b: number) {}
class	Blueprint for creating objects	class Person { name: string; }
interface	Defines object structure	interface User { name: string; age: number; }
unknown	Safer alternative to any	let value: unknown;
void	Represents no return value	function log(): void {}
union	Allows multiple types	let id: number

Syntax:

var **or** let **or** const **<**variable-name**> :** datatype **=** value**;**

Example:

var products : string = “mobiles”;

string:

It is collection of character is called as string.

We can create string in three ways:

1. “ ” (D ouble codes)
2. ‘ ’ (Single code)
3. ` ` (Backticks). It is used to define **paragraphs**. Introduced In **ES6 Version**.

Example:

var.ts

```
var subject : string = `Angular`;

var msg : string =`welcome to ${subject}`;
```

We can refer one variable to another variable. **`${variable-name}`.**

```
console.log(msg);
```

number:

```
var decimal : number = 100;

var double : number = 100.12345;

var hexadecimal : number = 0x123ABC;

var octal : number : 0o123;

var binary : number = 0b1010;

console.log(decimal, double, hexadecimal, octal, binary);
```

boolean:

- ❖ It has always true or false statements.

```
var isempty : Boolean = false;

var isabsent : Boolean = true;

console.log(isempty, isabsent);
```

union (|):

- ❖ It is the capable of two, more than different datatype values inside a single variable.
- ❖ Allows multiple types.

```
var my_var : string |number = “hello”;

my_var = 101;

console.log(my_var);    //  > 101;
```

number array:

Syntax:

var <variable-name> : datatype [] = [value1, value2, value3 …….];

var arr2: Array <number> = [value1, value2, value3 …….];

Example:

var rollnumbers : numbers[] = [1,2,3,4,5,6];

var empid : Array<number> = [s1, s2, s3, s4, s5];

Iterators:

1. forEach()
2. for….of()

var rollnumbers : numbers[] = [1,2,3,4,5];

var empid : Array<number> = [71, 72, 73, 74, 75, 76, 77, 78];

for the first iteration 1 values in the rollnumbers array stored into element.

And 0 index in the rollnumbers array will stored in the index value.

Its runs the loop until it will reach the last element.

Finally the element value contains the 1,2,3,4,5.

And also finally the index values contains the 0,1,2,3,4

```
rollnumbers.forEach((element :number, index :number)=>{
                    Console.log(element, empid[index]);
 })
```

empid = 71, 72, 73, 74, 75, 76, 77, 78.

Index of rollnumber = 0,1,2,3,4

So it prints the 71,72,73,74,75.

string array:

```
var courses: string[] = [`Angular`, `React`, `Vue.js`];

var AItools : Array<string> = [`Gemini AI`, `Claude AI`, `ChatGPT AI`, `Black box Ai`];
```

iteration

```
courses.forEach((ele:string, index:number)=>{
   console.log(ele,AItools[index])
});
```

Output:

angular Gemini AI

react Claude AI

Vue.js Chatgpt

Any:

- ❖ Allows any type (no type checking).
- ❖ We can store any type of datatype inside this.

Example:

```
Var apirequest: any =' We can store any type of data inside this.'

Console.log(apirequest);
```

JSON

1. JSON stands for JavaScript object notation.
2. JSON datatype is any.
3. JSON is used to transfer the data over the network.
4. JSON is light-weight.
5. JSON has contains the Key and value pairs and separated by “ : ”
6. And each key and values is separated by the (,).
7. The datatype of object is : { } Curly brackets.
8. The datatype of array is : [ ] Square brackets.

Example of JSON

```
var courses_list:any={
   "Subject_one": "Angular",
   "Subject_two": "Spring"
}

console.log(courses_list.Subject_one, courses_list.sub);   //  Angular Spring.
```

Functions

Function is a particular business logic .

Functions are used to reuse the business logic.

1) Named functions
2) Anonymous functions
3) Arrow function.

Named function

The function with user defined name called as named function.

Syntax:

Function definition

```
function function_name(parameter1:datatype,
                       parameter2:datatype,
                       parameter3:datatype, …… parameter n:datatype)
                      : returntype  {
business logic
}
```

Calling the function:

```
Function_name(argunment1, argunment2, ……..n);
```

Example with void :

```
function generatevalue() : void{
   console.log("Saigiri")
}
generatevalue();
```

Example with return statement:

```
function fun_one(name:string): string{
   // console.log(name)
   return name;
}
var res = fun_one("saigiri")
console.log(res)
```

Function Example with the string, undefined, null values:

```
function fun_two(param1:string, param2:string, param3:string):void{
   console.log(param1, param2, param3);
}
fun_two("Angular","Spring","Sql server");  //Angular Spring Sql server
fun_two(undefined, undefined, undefined);  //undefined undefined undefined
fun_two(null, null, null);  //null null null
```

For undefined and null it is showing error but it has executed the output.

Function with spread operator:

- In this parameter behave like a array. It will give output in the array form.

```
function spreadfun(...paramater:any):void{
   console.log(paramater)
}
spreadfun();
spreadfun(101)
spreadfun(null,null);
spreadfun(undefined,undefined)
spreadfun(1,2,3,4,5,6);
spreadfun("Sai","giri", "gowri")
```

- **Console:**

```
[]
[ 101 ]
[ null, null ]
[ undefined, undefined ]
[ 1, 2, 3, 4, 5, 6 ]
[ 'Sai', 'giri', 'gowri' ]
```

Note : Function won't accept more than 1 spread operator.

```
Function spread(…param1:any, …param2:any):void{
}
```

Note: Spread Operator should always should be in the last occurrence.

Otherwise it will through an error.

```
function fun(...para1 :any, para2:string){
}
```

A rest parameter must be last in a parameter list.

ES6

Default parameter in Functions:

```
function Default(name :string="saigiri"):void{
   console.log(name)
}
Default("Hello") // Hello
Default(undefined); //Saigiri
Default(null);     // null
```

Optional Parameters (?):

We will represent optional parameters with (?)

Optional parameters will introduced in ES6.

Final Named Function Example:

```
function finalfunction(param1:string, param2:string="saigiri", param3?:string, ...param4:any ){
   console.log(param1, param2, param3, param4)
}
```

//finalfunction(); //Expected at least 1 arguments, but got 0.

finalfunction("saigiri") //saigiri saigiri undefined []

finalfunction("saigiripeta", "Hello Angular",   "", 1,2,3,4,); //saigiripeta Hello Angular [1, 2, 3, 4]

finalfunction(undefined, undefined, undefined, undefined); //undefined saigiri undefined [undefined]

finalfunction(null, null, null, null); //null null null [null];

Function Hosting:

Calling the function before declaration called as function hosting.

We can overcome the function hoisting with the Anonymous function.

Anonymous Function.

The function without name is called as anonymous function.

We can overcome the "**function hosting**" with the help of Anonymous function.

We can store anonymous function to a variable.

Example:

```
let funvalue= function(){
console.log("Welcome")
}
funvalue();
```

JSON with an anonymous function Example:

```
let obj:any = {
    key1 : "",
    key2 : function(){
        let inner = function(){
            this.key1 = "ReactJS with TypeScript";
        }
        inner();
    }
};
obj.key2();
console.log( obj.key1 );
```

JSON with an anonymous function Example:

```
let obj:any = {
    key1 : "",
    key2 : function(){
        let inner = ()=>{
            this.key1 = "ReactJS with TypeScript";
        }
        inner();
    }
};
obj.key2();
console.log( obj.key1 );
//ReactJS with TypeScript
```

Anonymous functions breaks the parent scope

Arrow functions follows the parent scope.

Arrow Functions:

Arrow functions introduced in ES6 Version.

We will represent arrow functions with **"=>".**

Arrow functions also called as **'short hand'** functions

Arrow function, we use like **"callback"** functions.

Syntax:

```
()=>{}
Let fun_one (param1:datatype, param2:datatype):returntype =>{
//business logic.
}
Fun_one(1,2);  //calling the arrow function.
```

Example 1:

```
Let fun_one = ():string=>{
Console.log("Hello Typescript");
}
fun_one();
```

Example 2:

```
Let fun_two():string=>{
return "welcome to angular"
}
Let res:string = fun_two();
Console.log(res);
```

Example 3:

```
Let fun_three()=> "Angular with Typescript";
Let res = fun_three();
Console.log(res);
```

Example 4:

```
Let fun_four = (param1:string):string=>{
retrun “welcome to ${param1}”;
}

Let res1:string = fun_four(“angular 15”);
Console.log(res1);
Let res2:string = fun_four(“spring boot”);
Console.log(res2)
```

OOPs:

1. Class
2. Object
3. Inheritance
4. Polymorphism
5. Encapsulation
6. Abstraction

Class:

A collection of variables and methods is called a class

(Or)

Encapsulation of variables and methods is called as class.

We can create the classes by using “**class”** keyword.

We can create class with the help of **“new”** keyword.

We can refer class members with help of **“this”** keyword.

Typescript supports the 3 modifiers

1. Public
2. Private
3. Protected

Public modifiers recommended to functions

Private modifiers recommended to Variables.

Example of Class with variables and function

```
class Homepage{
  private data:string="Hello";

  public fun_one():string{
      return this.data;
   }
}
let obj = new Homepage();
console.log(obj.fun_one())
```

Example with default constructor:

```
class constr{
   products:string;
   images:string;
   flags:string
   constructor(){
      this.images="india map"
      this.flags="india",
      this.products="Flag cloth"
   }
}
```

```
let obj1 = new constr();
console.log(obj1.images,obj1.flags, obj1.products)
```

Example with paramatrized constructor:

```
class param_constructor{
    data1:string;
    data2:string;
    data3:string;
    constructor(arg1:string, arg2:string, arg3:string){
      this.data1 = arg1;
      this.data2 = arg2;
      this.data3 = arg3;
    }
}
let objpc = new param_constructor("Angular","Springboot","Sql server");
console.log(objpc.data1, objpc.data2, objpc.data3);
```

Example 2 with paramatrized constructor:

```
class tsclass{
   private data_one:any;
   private data_two:any;
   constructor(arg1:any,arg2:any){
      this.data_one =arg1;
      this.data_two =arg2
   }
   public function_one():any{
      return this.data_one;
   }
```

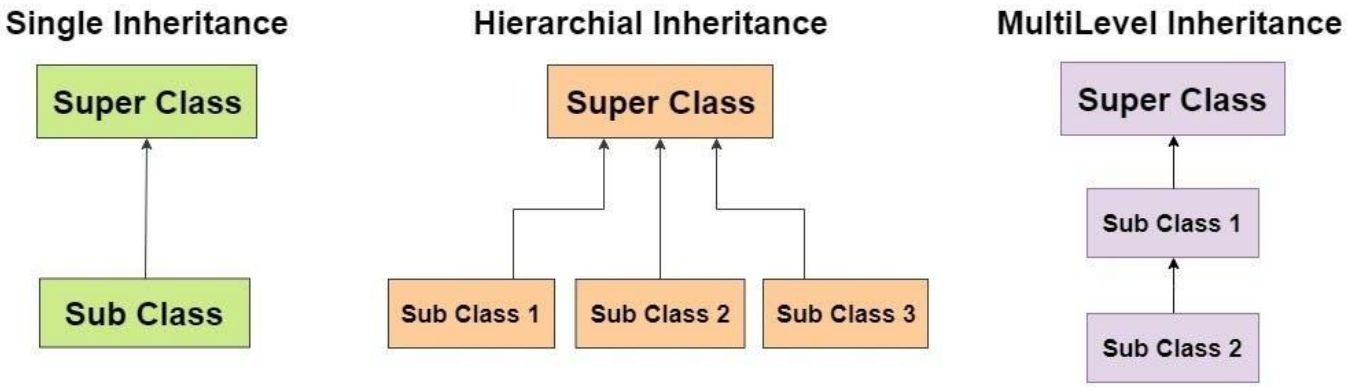
```
    public function_two():any{
        return this.data_two;
    }
}
let obj2 = new tsclass("Hello","Saigiri");
console.log(obj2.function_one())
console.log(obj2.function_two())
```

Note: We can access **"public arguments"** of constructor with class **"objects".**

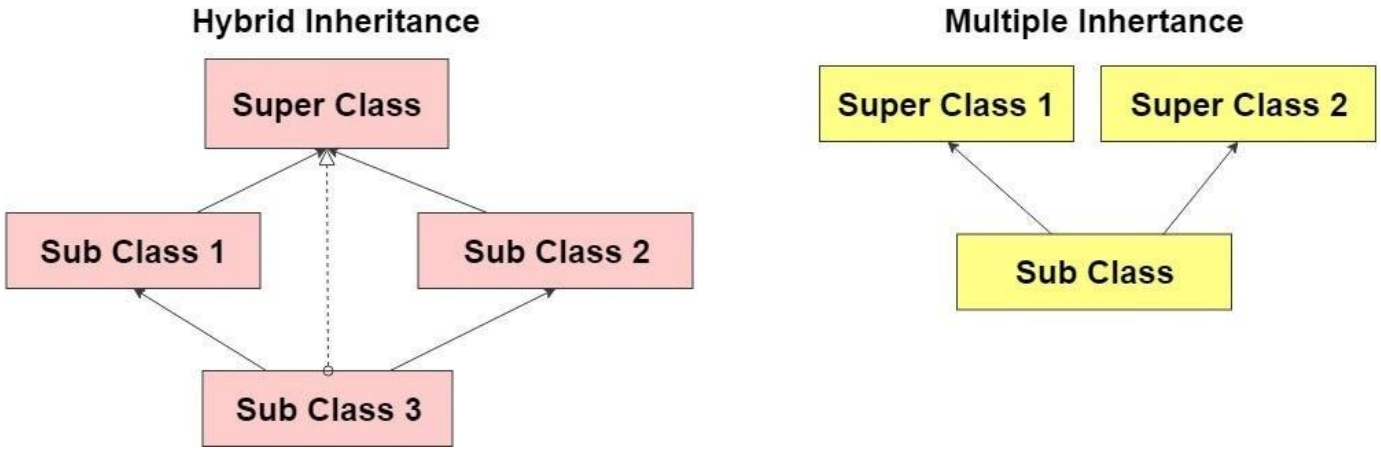
```
class class3{
    constructor(public arg1:any){
    }
}
let obj3:class3 = new class3("sai");

console.log(obj3.arg1)
```

Inheritance:

1. Getting the properties from parent class to child class called as Inheritance.
2. We will implement inheritance with the help of **"extends"** keyword.
3. TypeScript is a class-based, object-oriented language that explicitly supports two types of inheritance using the **extends** keyword: **Single Inheritance** and **Multilevel Inheritance**.
4. It does not support multiple or hybrid inheritance directly through classes.



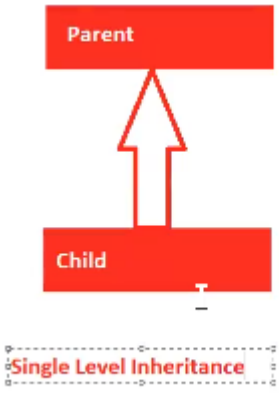
Hybrid and multiple inheritance is not supported in Typescript class.



We have 5 types of inheritance:

1. Single-Level Inheritance
2. Multi-Level inheritance
3. Multiple Level Inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance

Single Level Inheritance:

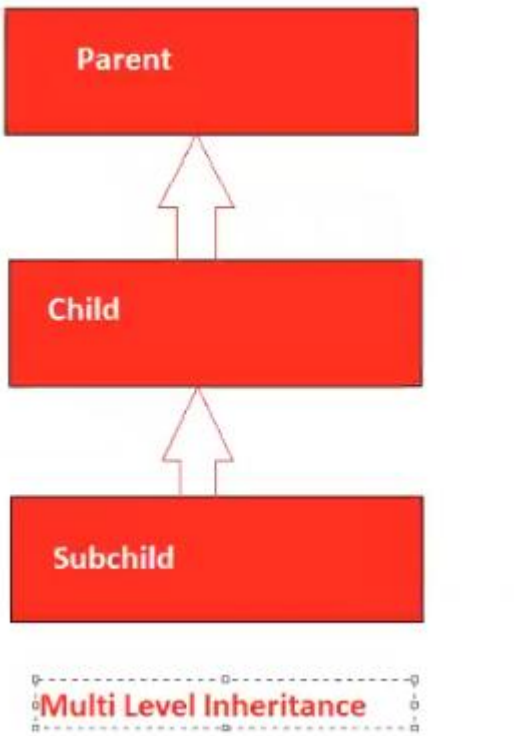


Example:

```
class Parent{
   Var_data: string ="Hello"
}

class child extends Parent{
   var_two :string ="Angular"
}
let p = new Parent();
console.log(p.Var_data)
let c = new child();
console.log(c.Var_data);
console.log(c.var_two)
let p1: Parent = new child();
console.log(p1.Var_data)

let c1: child= new Parent();
//Property 'var_two' is missing in type 'Parent' but required in type 'child'.
```

Multi-Level Inheritance:

- Multilevel inheritance in TypeScript allows a class to derive from another derived class, creating a chain (A ← B ← C) where C inherits properties/methods from B, which in turn inherits from A.
- It is implemented using the extends keyword, enabling code reusability across multiple levels.



1.Parent class

```
class Parent{
   fun_one(): string{
   return "Calling function one"
   }
}
```

2.Child class

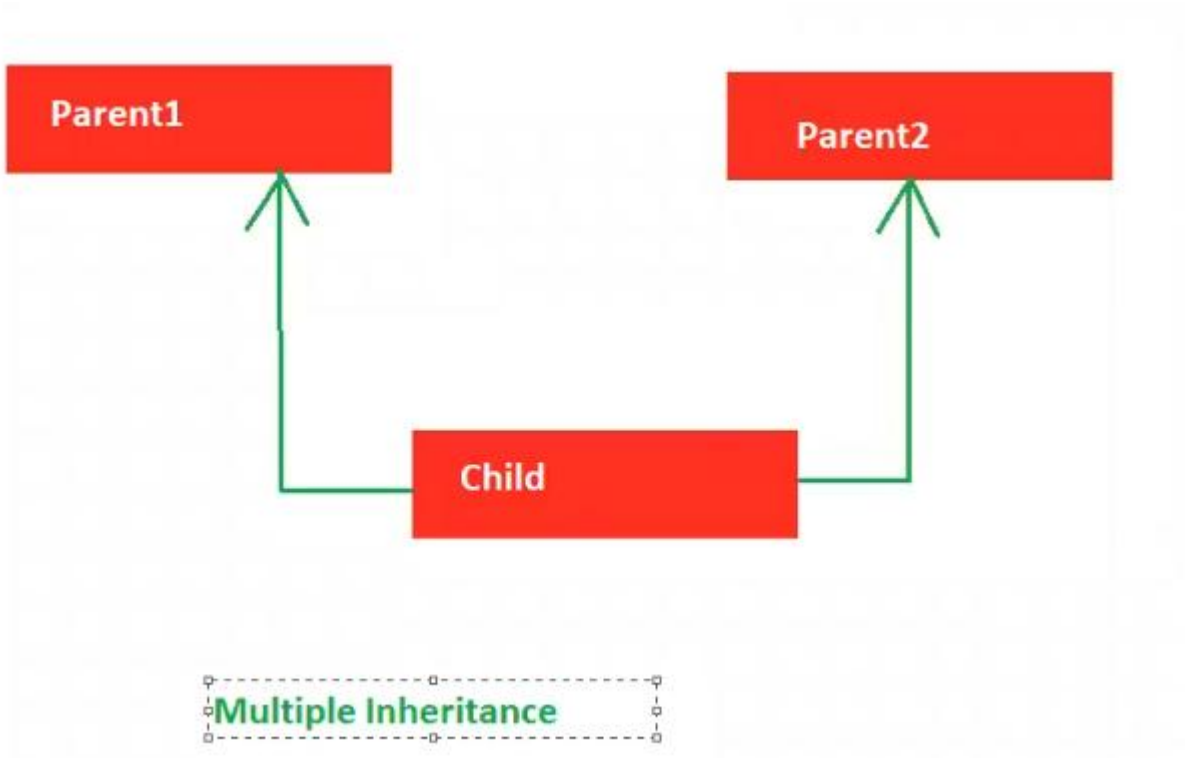
```
class child extends Parent{
   fun_two():string{
      return "Calling function two"
   }
}
```

3.Sub child class

```
class subchild extends child{
    fun_three():string{
        return "Calling function three"
    }
}
let obj  = new subchild();
console.log(obj.fun_one(), obj.fun_two(), obj.fun_three())
```

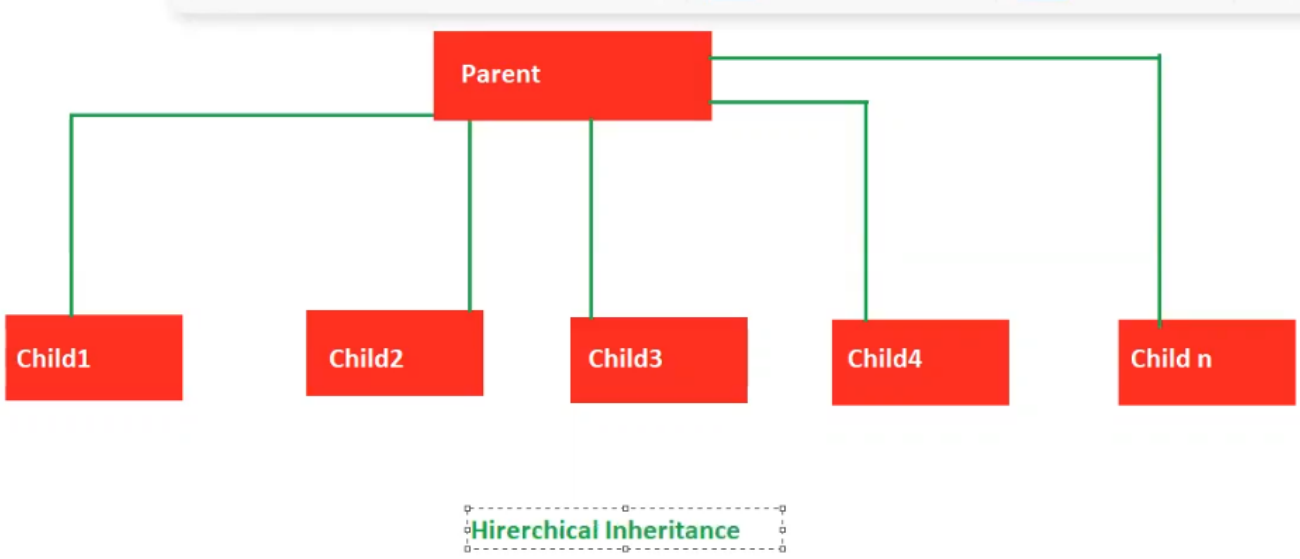
Multiple Inheritance:

- ❖ Multiple Inheritance is not supported in the typescript.



```
class Parent1{
    funPrint():string{
        return "Hello"
    }
}
class parent2{
    funDraw():string{
        return "World"
    }
}
class child1 extends Parent1, parent2{  //Classes can only extend a single class.
}
```

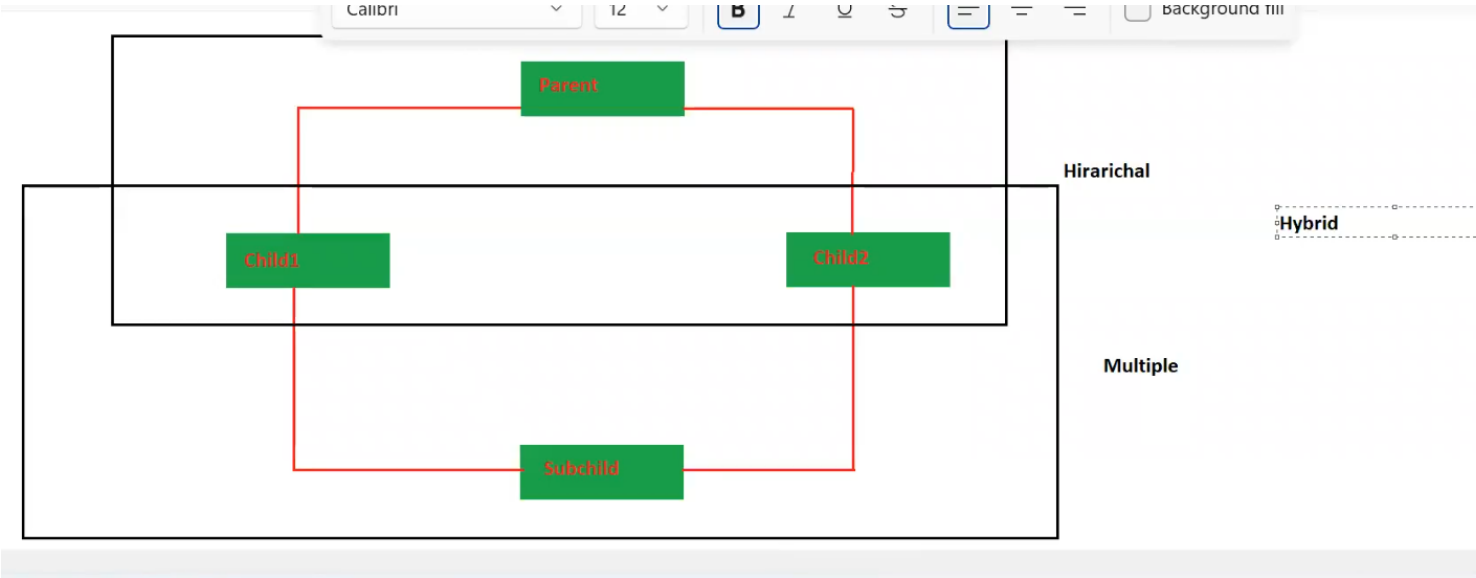
Hierarchical Inheritance:



Hirerchical Inheritance

```
class ParentOne{
 dialog1():string{
    return "Hello world"
 }
}
class child_one extends ParentOne{
 var_chone:string = "Child One"
}
class child_two extends ParentOne{
    var_chtwo:string="child two"
}

class child_three extends ParentOne{
    var_chthree:string ="child three"
}

let cobj = new child_one();
console.log(cobj.dialog1(), cobj.var_chone); //Hello world Child One

let cobj2 = new child_two();
console.log(cobj2.dialog1(),cobj2.var_chtwo) //Hello world child two
```

Hybrid inheritance:



class parenthy{}

class childhy1 extends parenthy{}

class childhy2 extends parenthy{}

class subhy extends childhy1,childhy2{} //Classes can only extend a single class.

Note: Hybrid inheritance is not supported by Typescript

Interfaces:

- Whenever we know only declarations, but we don't know implementations, then we will choose interfaces.
- We will create interfaces with the **"interface"** keyword.
- Implementations are provided by either **child classes or JSON**.
- Child classes provide implementations with the help of the **"implements"** keyword.

1. In TypeScript, an **interface** is a powerful feature used to define the "shape" or structure of an object, class, or function.
2. They act as a contract for type-checking during compilation, ensuring consistency and helping to catch errors early.
3. But they are purely a development-time tool and do not generate any JavaScript code at runtime.

Example 1:

```
// Define the shape of an object
interface User {
  id: number;
  name: string;
  email: string;
}
// Create an object that adheres to the User interface
const user: User = {
  id: 1,
  name: "Alice",
  email: "alice@example.com",
};
// Error: object must match the structure
// const invalidUser: User = { id: "2", name: "Jane" }; // Type error
```

Example 2:

```
interface interface1{
    Var_one:string;
}
class child implements interface1{
    Var_one: string ="welcome to interfaces"
}
let obj = new child();
console.log(obj.Var_one);
```

Example 3:

Interface with the JSON Object:

```
interface interface2{
   var_one:string;
}
let obj1: interface2={
   var_one : "interface with the object"
}
console.log(obj1.var_one); // interface with the object.
```

Example 4:

Interface with the JSON Object:

```
interface interface3{
   sub_one:string;
   sub_two:string;
   sub_three:string
}

let obj : interface3 ={
   sub_one:"hello",
   sub_two:"interface",
   sub_three:"Example"
}
console.log(obj.sub_one, obj.sub_two, obj.sub_three) //hello interface Example
```

Example 5:

Methods with the interfaces:

```
interface methinterface{
   fun_one():void;
   fun_two():void;
   fun_three():void;
}

let objmeth: methinterface ={
   fun_one : ():void=>{
      console.log("Calling method1")
   },
   fun_two :():void=>{
      console.log("Calling method2")
   },
   fun_three:():void=>{
      console.log("calling method3")
   }
}

objmeth.fun_one();
objmeth.fun_two();
objmeth.fun_three();
```

Single-level inheritance in interface:

```
interface interface11{
   product_one:string;
}
interface interface22 extends  interface11{
   product_two:string
}
let obj11 :interface22 ={
   product_one :"Example with the inheritance",
   product_two:"single level inheritance in the interface"
}
console.log(obj11.product_one, obj11.product_two)
```

Multi-Level inheritance in interface:

```
interface multi_face{
    multi_one : string
}

interface multi_child extends multi_face{
  multichild:string;
}

interface multi_super_child extends multi_child{
    super_child:string;
}

let mobj : multi_super_child={
    multi_one:"Multi Parent one",
    multichild:"Mutli child one",
    super_child:"Superchild"
}

console.log(mobj.multi_one, mobj.multichild, mobj.super_child)
```

Multiple Inheritance In interface:

It is possible to do multiple inheritance in the interface.

```
interface multiple_interface{
    multiple_one():number;
}
interface multiple_interface2{
    multiple_two():number;
}

interface multiple_interface3 extends multiple_interface,multiple_interface2{
      multiple_three():number;
}

let multiple : multiple_interface3 ={
    multiple_one:  ()=>{
        return 100
    },
    multiple_two: ()=>{
        return 200
    },
    multiple_three:()=>{
   return 300
    }
}
console.log(multiple.multiple_one(), multiple.multiple_two(),
multiple.multiple_three())
```

Abstract Classes

If we know the partial implementations, then we will choose abstract classes.

“abstract” is the keyword to create the abstract classes.

Implementations provide by child classes.

We can’t create object for the abstract class and interfaces.

Example 1:

```
abstract class class_one{
  meth1():void{
   console.log("Method one")
  }

  abstract meth2():void;
}

class child extends class_one{
    meth2():  void{
      console.log("Abstract method calling")
   }
}

let obj = new child();
console.log(obj.meth1(), obj.meth2())
```

Example 2:

```
abstract class abs1{
   meth1():string{
      return "Meth1 Calling"
   }

   abstract meth2():string;
}

abstract class abschild extends abs1{
    meth2(): string {
       return "Meth2 calling"
    }

    abstract meth3():string;
}

class abschild2 extends abschild{
   meth3(): string {
      return "Meth3 calling"
   }
}

let obj1 = new abschild2 ();

console.log(obj1.meth1(), obj1.meth2(), obj1.meth3())
```

Example 3 with interface, abstract class and child class:

```
interface interface1{
   meth1():any;
}

abstract class inabstr implements interface1{
   meth1():any {
      return "Interface with the child class as a abstraction"
   }

   abstract meth2():any;
}

class child2 extends inabstr{
   meth2():any {
      return "===>This using the inteface and the abstraction "
   }
}

let iaobj = new child2();
console.log(iaobj.meth1(), iaobj.meth2())
```

Modifiers:

Access Modifier	Accessible within class	Accessible within Sub-class	Accessible externally via cla instance
Public	Yes	Yes	Yes
Protected	Yes	Yes	No
Private	Yes	No	No

Public Modifier:

Public modifier applicable to variables, functions and constructors.

Public modifier won't be applicable to classes.

By default class is having public modifier. But we can't place before the public modifier.

We can access public members by using class objects.

Public members are available to the child class by default.

Example:

```
class publicm {
   public var_one:string;
   public constructor(){
      this.var_one ="constructor with the Public modifier "
   }

   public meth1():string{
      return "Public method"
   }
}
```

```
class child extends publicm{}
let obj = new child();
console.log(obj.meth1())
```

Private Modifier:

Private modifier applicable to **“variables”**, “**functions”** and “**constructors”**.

We can’t create an object for the **“Private constructor”** class.

We can’t access private members with class objects.

Private members wont accessible to child class.

Example 1:

```
class privatem{
   private var_one="Hello";

   private meth1():string{
      return "any"
   }
}
let objm = new privatem();
objm.var_one; //var_one' is private and only accessible within class 'privatem
objm.meth1();  //meth1' is private and only accessible within class 'privatem';
```

Example 2:

```
class private2{
   private var_one :string ="Private varibale"
}

class childp extends private2{}
let objp1 = new childp();

objp1.var_one; //var_one' is private and only accessible within class 'private2'
```

Example 3 with the private constructor:

```
class pconstr{
   private constructor(){

   }
}
let objpc = new pconstr()
//Constructor of class 'pconstr' is private and only accessible within the class declaration.
```

Example 4 with this keyword :

```
class thisprivate{
   private var_one:string ="Hello";
   public var_two :string = this.var_one;
```

```
    private meth1():string{
        return "with the this Keyword"
    }

    public meth2(){
        return this.meth1()
    }
}
class childth extends thisprivate{}
let objth = new childth();
console.log(objth.meth2());
console.log(objth.var_two)
```

Protected Modifier:

- ❖ Protected modifier applicable to variables, functions and constructors.
- ❖ Protected members' availability to child classes.
- ❖ We can’t access we can’t access class objects.
- ❖ We can’t create an object in a protected constructor class.

Example:

```
class protectedM{
    protected Var_one:string ="hello";
    protected meth1():void{
        console.log("hello world")
    }
  protected constructor(){ }
```

```
}

class proctectedchild extends protectedM{}

let objpro = new proctectedchild(); // protected and only accessible within the class declaration.
```

Example with this keyword, child class:

```
class protected2{

    protected var_one :string ='Using this keyword'

}


class protectedchild extends protected2 {

   public var_two  = this.var_one;

}

let pc = new protectedchild();

console.log(pc.var_two) // Using this keyword
```

	Public	**Private**	**Protected**
Variables, methods And constructor.	Applicable	Applicable	Applicable
Accessible to child class	Yes	No	Yes
Accessible to child class **Objects.**	Yes	No	Yes
Constructor	We can create an object.	We **can't** create an object.	We **can't** create an object.

Polymorphism:

- ❖ It is a process of performing multiple tasks with the same identity.
- ❖ Behaves like many is called as polymorphism
- ❖ Polymorphism contains 2 forms
- ❖ “Run-time polymorphism” **(Method Overriding)**
- ❖ “Compile-time polymorphism“ **(Method Overloading)**

Method or Function Overriding :

1. Overriding parent class functionality with child class functionality is called as function overriding.
2. Writing two or more methods in two different classes having the same method name, Same parameters, and same return type is called as **run-time polymorphism**.
3. The method present in super class is called an **overridden method**.
4. When an overridden method is called through a super or parent class.
5. To implement method overriding, inheritance is mandatory.
6. Both parent class and child class should contains same function.

Example:

```
class Parent{
   dbfun():string{
      return "Data comes from the Oracle Database" }   }
class child extends Parent{
   dbfun(): string {
      return "Data comes from the SQLite Database"  }   }
let obj = new child();
console.log(obj.dbfun())
```

Method or Function Overloading :

1. Same function name and same number of parameters, but different datatypes are called as function overloading.
2. It occurs within the **same class**
3. Writing two or more methods in the same classes having the same method name, same return type but different parameters is called as **Compile time polymorphism.**
4. Inheritance is not involved since it deals with only one class.
5. In Overloading return type need not to be same.
6. Parameters must be different when we do overloading.
7. In overloading one method can't hide another method.

Example

```
class Parent1{
   meth1(name:string, initial:string):void;
   meth1(mobile:number, street:string):void;
   meth1(pincode:number, aadharno:number):void;

   meth1(address:any, income:any):void{
      console.log(address, income)
   }
}
let objol = new Parent1();
console.log(objol.meth1("saigiri","Peta"));
console.log(objol.meth1(709566656,"IBS"));
console.log(objol.meth1(500201,8797979));
//console.log(objol.meth1("Ahjje",7898789)); //No overload matches this call
```

Method Overriding	Method Overloading
1) It occurs in the parent and child classes.	1) It occurs within the same class
2) Inheritance is involved because it occurs between parent and child classes	2) Inheritance is not involved because it deals with only one class
3) In Overriding return type must be the same	3) In Overloading return type need not be same
4) Parameters must be the same	4) Parameters must be different when we do overloading.
5) In overriding, child class methods hides the superclass methods	5) In overloading, one method can't hide another method.
6) It is called as **run-time polymorphism**	6) It is called as **compile time Polymorphism**

Static Keyword:

- ❖ Static modifier applicable to variables and functions.
- ❖ Static members we can access directly by using the class name
- ❖ We need not create the object for the static functions.
- ❖ We can't refer to static variables and functions by using the **"this"** keyword.
- ❖ We can't initialize static members with a constructor.
- ❖ We can't access static members with class objects .

Example:

```
class static1{
 static var_one:string ="saigiri"
 static var_two:string="hello";
 static meth1c():void{
```

```
    console.log("meth1 calling")  }  }



let objs= new static1();

static1.meth1c();

 console.log(static1.var_two);

//objs.var_one;

//ar_one' does not exist on type 'static1'. Did you mean to access the static member 'static1.var_one' instead?
```

readonly:

readonly is the keyword in Typescript.

readonly is used to “read” the content, but we can’t update the content.

We can initialize readonly members with a constructor.

Example:

```
class readonly{
    readonly var_1 :string;
    constructor(){
        this.var_1 ="Readonly "
    }
}
let obj = new readonly();
console.log(obj.var_1);
obj.var_1 ="Hello" //Cannot assign to 'var_1' because it is a read-only property.
```

Super keyword:

Super keyword is used to call the **“parent class members”.**

Super keyword applicable to **“constructors”** and **“functions”.**

Example:

```
class superclass{
   constructor(name:string, age:number){
     console.log(`${name}, ${age}`)
   }
   meth2():string{
      return "Hello"
   }
}
class superchild extends superclass{
   constructor(name:string,age:number,pincode: number){
      super(name,age)
      console.log(`${name}, ${age},${pincode}`)
   }
   meth3() :string{
      return super.meth2();
   }
}
let sp2 = new superchild("saigiri",121,2)
console.log(sp2.meth2())
console.log(sp2.meth3())
```

Feature	this Keyword	super Keyword
Definition	Refers to the **current instance** of the class.	Refers to the **parent (base) class**.
Purpose	Used to access members (properties/methods) of the current class.	Used to access members or constructors of the parent class.
Constructor Usage	Used to initialize properties of the current object.	**Must** be called as super() before using this in a subclass.
Method Calling	Calls the version of a method defined in the **current** class.	Calls the version of a method defined in the **parent** class.
Scope	Limited to the current class and its inherited members.	Limited to the immediate parent class.
Real-world Analogy	"My own unique features" (e.g., My height).	"My family's features" (e.g., My father's surname).

Modules:

A collection of related data/a group of logical data is called a module.

A Collection of modules is called as Project.

“exports” and default are the keywords used to export the modules.

“import” is the keyword used to import the module.

If we export the module, it will be stored as an object {}.

So at that time importing we use this object {} in the import module.

And also we will export using the **“default”** keyword and it will not stored as a object.

In the importing the module we will not use any object to import .

We use directly with the help of name.

If we want to run the module there is one command

- : tsc - -module common.js <modulename.ts>
- : Ex tsc - -module common.js <first.ts>
- To run in JS : node first.js

Here - - module is the flag

Common.js is the library.

module1.ts > default

```ts
export  let var_one :string ="Hello";


let name:string ="saigiri peta"

export default name;
```

module2.ts

```ts
import { var_one } from "./module1";

import name from "./module1"

console.log(var_one);

console.log(name)
```

We can export all variables at a time.

And also we can import all variables at a time.

```ts
let name1: string ="Sai";
let name2:string ="giri";
let name3 :string ="Peta";

export {name1,name2,name3};
```

```ts
import * as myvar from "./module1";

console.log("====>"+myvar.name1, myvar.name2, myvar.name3)
```

Example with Function using export and import :

module1.ts > ...

```ts
function fun_one(){
    console.log(`Function calling with
        the export and imports`)
}

export default fun_one;
```

TS module2.ts

```ts
import fun_one from "./module1";

console.log(fun_one())
```

Example with the class using export and import:

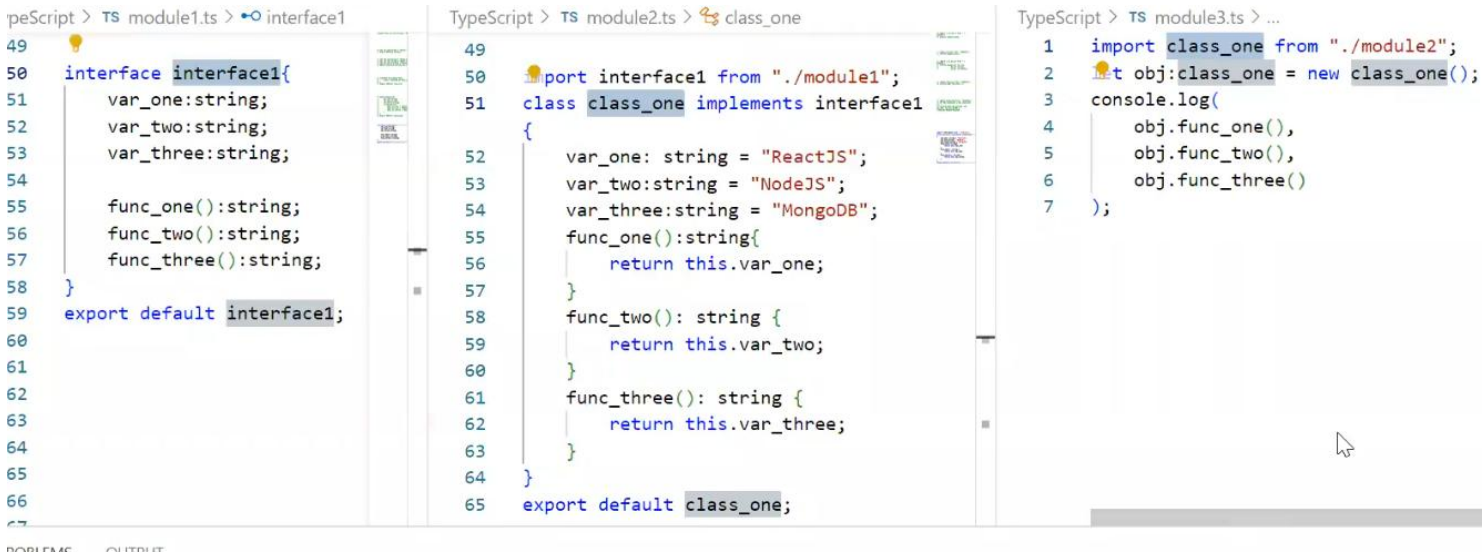
classexport.ts > default

```ts
class class1{
    var_one:string;
    var_two:string;
    var_three:string;

    constructor(){
        this.var_one="one";
        this.var_two="two";
        this.var_three="three"
    }
}

export default class1;
```

TS classimport.ts > ...

```ts
import class1  from "./classexport";

let obj = new class1();
console.log(obj.var_one, obj.var_two, obj.var_three)
```

Example with the class using export and import:

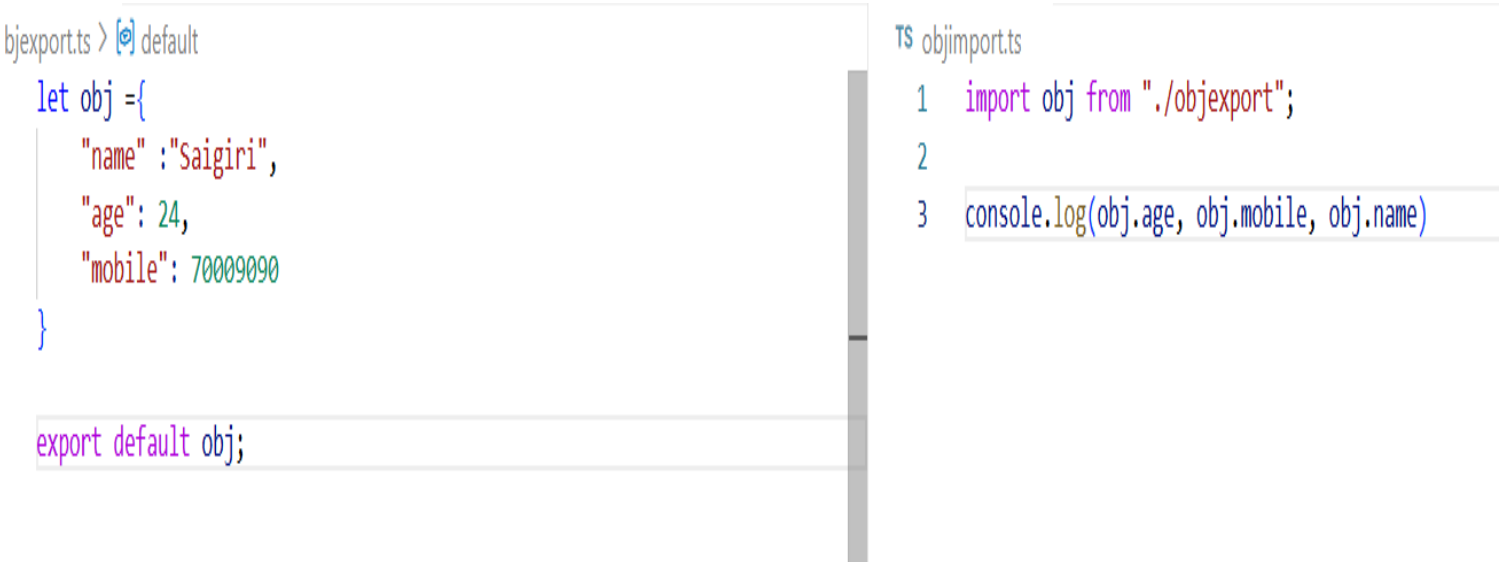


```
// module1.ts
interface interface1{
    var_one:string;
    var_two:string;
    var_three:string;

    func_one():string;
    func_two():string;
    func_three():string;
}
export default interface1;
```

```
// module2.ts
import interface1 from "./module1";
class class_one implements interface1
{
    var_one: string = "ReactJS";
    var_two:string = "NodeJS";
    var_three:string = "MongoDB";
    func_one():string{
        return this.var_one;
    }
    func_two(): string {
        return this.var_two;
    }
    func_three(): string {
        return this.var_three;
    }
}
export default class_one;
```

```
// module3.ts
import class_one from "./module2";
let obj:class_one = new class_one();
console.log(
    obj.func_one(),
    obj.func_two(),
    obj.func_three()
);
```

Example with the object:



```
// bjexport.ts
let obj ={
    "name" :"Saigiri",
    "age": 24,
    "mobile": 70009090
}

export default obj;
```

```
// objimport.ts
import obj from "./objexport";

console.log(obj.age, obj.mobile, obj.name)
```

Generics

Generics are used to achieve "**strict typing**"

Generics are also used to decide datatype dynamically.

Example 1:

```
function fun_one<A,B>(param1:A, Param2:B):void{
    console.log(param1, Param2)
```

```
}
fun_one<string, number >("Hello", 28);
```

Example 2 with class:

```
class Generics<A,B,C>{
   product1:A;
   product2:B;
   product3:C;
   constructor(arg1:A, arg2:B, arg3:C){
      this.product1 = arg1;
      this.product2 = arg2;
      this.product3 = arg3
       console.log(this.product1, this.product2 ,this.product3)
   }
}
let obj = new Generics<string, number, any>("Generics", 12, "Example")
console.log(obj.product1)
```

Example 3 with class inside the funtion:

```
class genfun{
   funwithgenerics<A,B>(param1:A,param2:B):void{
      console.log(param1, param2)
   }
}
let obj1= new genfun();
```

```
console.log(obj1.funwithgenerics<string,string>("====>"+"Function with",
"Generics"))
```

Example 4 with Interface with object and also child class:

```
interface Interface1<A,B>{
   product1:A;
   product2:B;
}
let obj2 :Interface1<number,number>={
   product1 : 120,
   product2 :130
}
console.log(obj2.product1);
console.log(obj2.product2)

class child implements Interface1<any, any>{
   product1: any="Pen Stand";
   product2: any ="Book";
}

let ch = new child();
console.log(ch.product1, ch.product2)
```

JSON

1. JSON stands for **JavaScript object notation**.
2. JSON is also called as "**JavaScript objects**"
3. JSON datatype is any.
4. JSON is used to transfer the **data over the network**.
5. JSON is "**light-weight"** .
6. Light-weight Means consumes less internet and create less burden on hardware devices.
7. JSON has contains the Key and value pairs and separated by " : "
8. And each key and values is separated by the (,).**Data** ----------key & Value pairs
9. The datatype of object is : { } Curly brackets. **Objects** ---------- { }
10. The datatype of array is : [ ] Square brackets. **Arrays** ----------[]

Example of JSON

```
var courses_list:any={
   "Subject_one": "Angular",
   "Subject_two": "Spring"
}
```

Example 1 with html code :

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>JSON</title>
</head>
<body>
   <!-- <script>
```

```
    let obj = {
        name : "saigiri",
        mobile :7095997321,

    }
    document.write(obj.mobile, obj.name)
</script> -->

<!-- <script>
    let obj = {
        key1: {
            msg: "Welcome to JSON"
        }
    }
    document.write(obj.key1.msg)
</script> -->

<script>
    let json3 = {
        key:"Mean Stack",
        key1:{
            Front_End :"Angualr 20"
        },
        key2:{
            Backend:"Spring Boot"
        },
        key3:{
            Database:"SQL Server"
```

```
        }
    }
    let value1 = json3.key;
    let value2 = json3.key1.Front_End;
    let value3 = json3.key2.Backend;
    let value4 = json3.key3.Database;

    document.write(`<table border="1" align="center" cellspacing= "10px" cellpading="10px" border="black" border-radius="5px">
        <tr>
            <td colspan="3"> ${value1} </td>
            </tr>
            <tr>
                <td> ${value2} </td>
                <td> ${value3} </td>
                <td> ${value4} </td>
                </tr>
                </table>`)
  </script>
</body>
</html>
```

Example 2 with html code :

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>JSON2</title>
</head>
<body>
   <script>
      let obj = {
         key1:{
            fullstack:"MEAN STACK"
         },
         key2:{
            frontend:"Angular"
         },
         key3:{
            backend:"Spring Boot"
         },
         key4:{
            database:"SQL Server"
         }
      }

      document.write(`<table border="1px" border-radius="5px" align="center"
cellspacing="10px" cellpadding="10px" >

         <tr >
```

```
            <td colspan="1"> ${obj.key1.fullstack}</td>
            <td> ${obj.key2.frontend}</td>
            <td> ${obj.key3.backend}</td>
            <td> ${obj.key4.database}</td>
            </tr>
            </table>`)
    </script>
</body>
</html>
```

Example 3 with html code :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JSON Example 3</title>
</head>
<body>
    <script>
        let obj =[
            {p1:1,pname:"pone",pcost:100},
            {p1:2,pname:"ptwo",pcost:200},
            {p1:3,pname:"pthree",pcost:300},
```

```
        {p1:4,pname:"pfour",pcost:400},
        {p1:5,pname:"pfive",pcost:500}
    ];
    obj.forEach((ele,ind)=>{
        document.write(`<table align="center" cellspacing="10px" cellpadding="10px" border="1px solid black">
            <tr>
                <td > ${ele.p1} </td>
                <td > ${ele.pname} </td>
                <td > ${ele.pcost }</td>
                </tr>
                </table>`)
    })
  </script>
</body>
</html>
```

Destructuring Example 4 with html code :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Destructring JSON ES6</title>
</head>
<body>
  <script>
```

```
        let obj ={
            key1:"Hello",
            key2:"Hello2",
            key3:"Hello3"
        }
        // let j = JSON.stringify(obj)
        // document.write(j)
        let {key1,key3,key2} = obj;

        document.write(key1,key2,key3)
    </script>
</body>
</html>
```

Destructuring Example 5 with html code :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Destructuring 2</title>
</head>
<body>
    <script>
        let obj ={
            key1:{
```

```
            key2:"Destructuring"//
          }
        }

        let {key1} = obj;
        let {key2} = key1;
        document.write(key2)
          </script>
</body>
</html>
```

JSON Rearrangement Example 6 with html code :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JOSN Renaming with :</title>
</head>
<body>
    <script>
        let obj = {
            hyd:{
                Edu:"Ameerpet"
            },
```

```
        vizag:{
            Edu:"gayatri Collage"
        },
        Srikakulam:{
            Edu:"Sun degree collage"
        }
    }

    let {hyd,vizag,Srikakulam} = obj;
    let {Edu : HydEdu} = hyd;

    let {Edu: vizedu} = vizag;

    let {Edu: sklmedu} = Srikakulam

    document.write(HydEdu,vizedu,sklmedu)
  </script>
</body>
</html>
```

JSON Duplicates Example 7 with html code :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Duplicate keys in JSON</title>
</head>
<body>
    <script>
        let obj = {
            key1:100,
            key1:200
        }
         let {key1} = obj
        document.write(key1)
        //old value replaced with new value.
    </script>

    <script>
        let obj1={
            key3:"Hello1",
            key4:"Hello1"
        }
        let {key3,key4}=obj1;
        document.write(key3,key4)
    </script>
```

```
</body>
</html>
```

JSON Copy Example 8 with html code :

Taking the xerox copy is called “**Immutability”.**

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Copying JSON Data</title>
</head>
<body>
    <script>
        // Taking the xerox copy is called Immutability
        let obj ={
            key :"Angular 16"
        }
        let xerox ={...obj};
         obj.key ="Angular with Typescript";
         xerox.key= "Spring boot with microservices";
         let {key : original}= obj;
         let {key : duplicate}= xerox;
         document.write(original);
         document.write(duplicate)
```

```
    </script>
</body>
</html>
```

Multiple JSON Copy Example 9 with html code :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JSON Multiple copies</title>
</head>
<body>
    <script>
        let obj ={
            key1:"Angular",
        }

        let obj1={
            key2:"Spring Boot"
        }

        let obj2 ={
            key3:"Sql Server"
        }
```

```
        let obj4 ={...obj,...obj1,...obj2};


        let {key1,key2,key3} = obj4;


        document.write(`<p>${key1}</p>
        <p>${key2}</p>
        <p>${key3}</P>`)
    </script>
</body>
</html>
```

Delete JSON Keys Example 10 with html code :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Delete JSON Keys</title>
</head>
<body>
    <script>
        let obj={
            key1:"Hello Wolrd",
```

```
        key2:"Hello sai"
      }
      delete obj.key1;
      let {key1,key2}=obj;
      document.write(key1,key2)
   </script>

</body>
</html>
```

Typeof method and convert object into string with Example 11:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Knowing the JSON DataType of Object</title>
</head>
<body>
   <script>
      let obj = {
         key1:"Hello"
      }
      document.write(typeof obj); //object
    let str=   JSON.stringify(obj)
```

```
    document.write(typeof str) // string
   </script>
</body>
</html>
```

Square Operator [] in JSON with Example 12:

The (.) operator is only allowed for the string only.

To Overcome this their introduced the Square operator [].

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Square Operator</title>
</head>
<body>
   <script>
      let obj ={
         key1:"Hello",
         100:'hello2',
         true:"False"
      }
      document.write(obj.key1);
  document.write(obj[100]);
  document.writeln(obj[true])
   </script>
</body>
</html>
```

To get the JSON Keys and Values separate by using Object.keys() and Object.values() methods

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>To get the keys of JSON </title>
</head>
<body>
    <script>
        let obj ={
            key1:"Hello1",
            key2:"Hello2",
            key3:"Hello3"
        };
        let  keys = Object.keys(obj)
        // document.write(keys)
        // ["key1","key2","key3"]
         let values = Object.values(obj);
        document.writeln(values)
        if(keys.length !=0){
            document.write("the object is not empty")
        }else{
            document.write("Object is empty")
        }
    </script>
</body>
</html>
```

Map Method with Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Map Method with Examples</title>
</head>
<body>
    <script>
        let arr =[1,2,3,4,5];
    let newarr = arr.map((ele,ind)=>{
            return ele*100;
        })
        document.writeln(newarr)
        console.log(newarr)
    </script>
</body>
</html>
```

Map Method with Example 2:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Map methpd with example</title>
</head>
<body>
    <script>
```

```
      let obj
=[{"key1":"hello1"},{"key1":"hello2"},{"key1":"hello3"},{"key1":"hello4"},{"key1":"hello5"}];


 let newobj =     obj.map((ele,ind)=>{
         if(ind === 0 ){
            return {...ele,"key2":"welcome1"}
         }else if(ind === 1){
            return {...ele,"key2":"Welcome2"}
         }else if(ind ===2){
            return {...ele,"key2":"Welcome3"}
         }else if(ind ===3){
            return {...ele,"key2":"welcome4"}
         }else if(ind ===4){
            return {...ele,"key2":"welcome5"}
         }else{
            return {...ele,"key2":"Welcome"}
         }
      })
//convert this into switch case
      console.log(newobj)
   </script>
</body>
</html>
```

Filter with the JSON Example 1:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
```

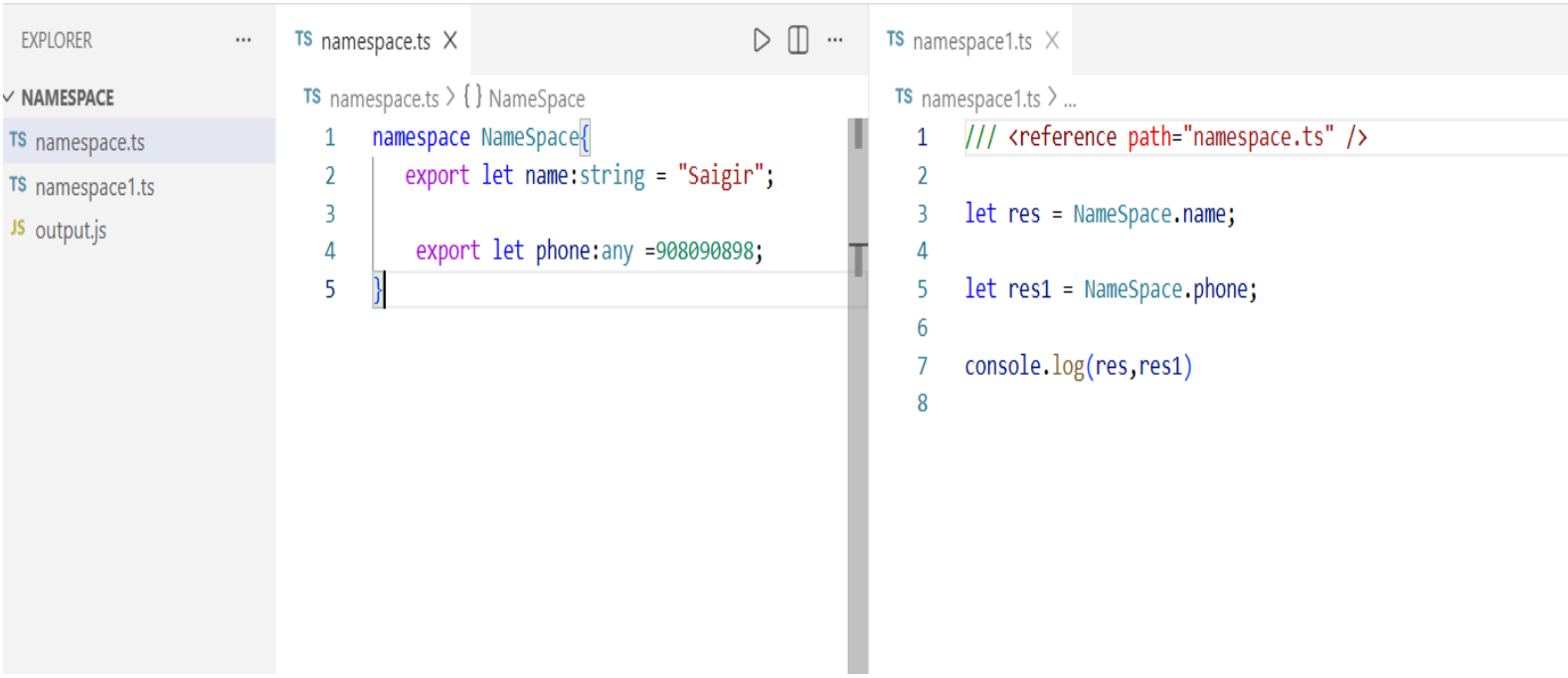
```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Filter Method</title>
</head>
<body>
    <script>
        let arr =[10,20,30,40,50];
      let newarr=  arr.filter((ele,ind)=>{
            return ele>=30;
        })
        document.write(newarr)
    </script>
</body>
</html>
```

Filter with the JSON Example 1:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Filter with the example</title>
</head>
<body>
    <script>
    let arr = [{eid:111,esal:10000,ejob:"sales"},
    {eid:222,esal:1000,ejob:"Sales boy"},
    {eid:333,esal:20000,ejob:"manager"},
    {eid:444,esal:30000,ejob:"CEO"},
    {eid:555,esal:40000,ejob:"Developer"}]
```

```
  let newarr=  arr.filter((ele,ind)=>{
    if(ele.esal >= 30000){
       return ele.ejob
    }
    else if(ele.esal <= 2000){
       delete arr.ele
    }
    })
    document.writeln(newarr)
    console.log(newarr)
    </script>
</body>
</html>
```

Namespace



To compile the namespace :

- tsc namespace1.ts --outFile output.js

To run the namespace

- node output.js

Tuple is a heterogeneous array contains predefined length and type for each index.

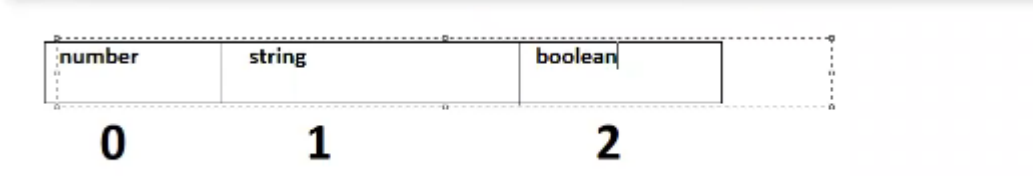
Heterogeneous means the different datatype. And also it is fixed length.

Each index contains different datatype.

We can increase the size of the tuple by using the push() method.

If required we can restrict the size also. Using the readonly keyword.

And also we can decrease the size of the tuple by using pop() method.



Example:

```
let var1 :[number,string, boolean];
var1 =[100,"Angular",true]
var1.forEach((ele,ind)=>{
    console.log(ele)
})
let var2 :[number,number];
var2=[100,200];
var2.push(300)
console.log(var2)
let var3 :readonly[number,number];
var3=[100,200];
var3.push(300)
//Property 'push' does not exist on type 'readonly [number, number]'
//Destructuring the Tuple
let var4 : [number, number];
var4=[100,200];
let [x,y]= var4;
console.log(x);
console.log(y);
```

Destructuring tuple with Example:

```
let var5 :[x1:number,y1:number];
var5 =[10,20];
let [x1,y1]= var5
console.log(x1,y1)
```

Enum

Enum is used to create custom constant

“**enum**” is a keyword.

And also we can create our own values for the enum.

Example 1:

```
enum Enum{
   CONST1,
   CONST2,
   CONST3,
   CONST4
}
console.log(Enum.CONST1, Enum.CONST2, Enum.CONST3, Enum.CONST4)  // 0 1 2 3
//Enum.CONST1 =100;
//Cannot assign to 'CONST1' because it is a read-only property.
```

Example 2:

```
enum Enum1{
   x=1,
   y,
   z
}
console.log(Enum1) //{ '1': 'x', '2': 'y', '3': 'z', x: 1, y: 2, z: 3 }
```

Example 3:

```
enum Enum3{
   const =100,
   const1 =200,
   const2 =300
}

console.log(Enum3.const, Enum3.const1, Enum3.const2) //100 200 300
```